

claude-windows / 7fce5410-4c57-4ec9-a054-be898bda22da

Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\7fce5410-4c57-4ec9-a054-be898bda22da\subagents\workflows\wf_c81dbe7a-7ff\agent-ab4edee4dda38386c.jsonl`
- SHA-256: `52b41c3420b832ea0a68779f4218b7c7168a40222483998ab9a1e4ff38e814e4`
- Source modified: `2026-06-25T16:56:59+00:00`
- Imported at: `2026-07-05T16:48:26+00:00`
- project: `wf_c81dbe7a-7ff`
- session_id: `7fce5410-4c57-4ec9-a054-be898bda22da`
```

Transcript

1. user (2026-06-25T16:54:30.948Z)

PR #380 "feat(01/P11): Gemini remote-file GC ? startup orphan sweep" is CHECKED OUT (branch 01-p11/gemini-remote-file-gc). It adds a best-effort startup sweep that deletes orphaned Gemini File-API uploads after a hard crash. Changed files (read the real diff with: git diff origin/master...HEAD): backend/providers/gemini.py (new method GeminiProvider.cleanup_orphaned_files(grace_sec=3600) using client.files.list()/delete()); backend/main.py (new _maybe_sweep_gemini_orphans(config), wired in main() next to fail_stale_jobs()); tests/test_provider_gemini.py (+6 tests). CI is GREEN ? do NOT re-run CI/ruff/mypy/tests; review the CODE for correctness, safety, and conventions. Read the actual files + surrounding code to ground every claim.

Review ONLY the SAFETY of deleting remote files. Central risk: could the sweep delete a file a legitimate LIVE process still needs? Consider: (a) a single upload whose processing/generate cycle exceeds grace_sec (1h) ? long video or many retries ? then a concurrent process's startup sweep deletes it mid-use; how likely, and what breaks? (b) multiple concurrent worker processes can run (the repo's single-executor guard is per-process) ? does the 1h grace actually prevent racing a live upload, or just narrow it? (c) what create_time represents and whether 1h is a safe default; (d) files.list() runs on EVERY startup ? boot latency/cost/failure-mode impact. Report findings with severity + a fix. Be skeptical and precise.

2. assistant (2026-06-25T16:54:33.402Z)

I'll review the safety of the Gemini remote-file GC sweep. Let me start by reading the actual diff and the surrounding code.

3. user (2026-06-25T16:54:35.168Z)

```
backend/main.py          | 25 ++++++++
backend/providers/gemini.py | 43 +++++
tests/test_provider_gemini.py | 104 +++++
3 files changed, 172 insertions(+)
---HEAD---
f00b71e feat(01/P11): Gemini remote-file GC ? startup orphan sweep
86705c1 chore: ruff format gemini_refine.py + trajectory_hybrid.py (#379)
b72ac85 fix(B2/P17): surface Gemini provider failures in all callers (#378)
```

4. assistant (2026-06-25T16:54:37.353Z)

Let me read the actual diff for the two source files.

5. user (2026-06-25T16:54:38.275Z)

```
diff --git a/backend/main.py b/backend/main.py
index 5561cee..ea5a875 100644
```

```

--- a/backend/main.py
+++ b/backend/main.py
@@ -64,6 +64,7 @@ from backend.pipeline.segmenters.base import segmenter_registry
    from backend.pipeline.stitcher.compiler import VideoCompiler
    from backend.pipeline.stitcher.watermark_i18n import localize_watermark
    from backend.pipeline.validators.base import registry
+from backend.providers import provider_registry
    from backend.reporting import emit_indexer_report
    from backend.results import Failure # standardized error/result contract
    from backend.utils.ffmpeg import ( # P2a: single ffmpeg/ffprobe resolver
@@ -1184,6 +1185,25 @@ def _enforce_startup_or_exit(config) -> None:
    sys.exit(1)

+def _maybe_sweep_gemini_orphans(config) -> None:
+    """O1/P11: best-effort sweep of Gemini File-API uploads orphaned by a previous crashed run.
+    A no-op unless the configured provider is Gemini AND its API key is present. Never raises ?
+    a failed sweep is logged and swallowed so it can't block startup. Extracted for unit
    tests."""
+    try:
+        provider_name = getattr(config, "ai_provider", "")
+        if provider_name != "gemini":
+            return
+        api_key = os.environ.get("GEMINI_API_KEY")
+        if not api_key:
+            return
+        provider = provider_registry.get("gemini", api_key=api_key, model_name=config.ai_model)
+        deleted = provider.cleanup_orphaned_files()
+        if deleted:
+            logger.info("[STARTUP] Gemini orphan sweep reclaimed %d remote file(s).", deleted)
+    except Exception as e:
+        logger.warning("[STARTUP] Gemini orphan sweep skipped (non-fatal): %s", e)
+
+def main():
+    parser = argparse.ArgumentParser(description="Sports Highlight Indexer Orchestrator")
+    parser.add_argument("--video-path", help="Local path to the MP4 sports video file")
@@ -1319,6 +1339,11 @@ def main():
    if swept:
        logger.warning(f"[JOBS] Swept {swept} stale job(s) from a previous run ? failed.")

+    # O1/P11: sweep Gemini File-API uploads orphaned by a hard process death (the per-call
+    # _safe_delete covers normal/except paths, but a SIGKILL/OOM mid-upload leaves a clip on
+    # Google's servers forever). Best-effort, video-only, grace-gated; never blocks startup.
+    _maybe_sweep_gemini_orphans(global_config)
+
+    # CLI processing mode (no web UI needed). --app/--server always run the server, never CLI
    single-shot.
    if args.video_path and not args.server and not args.app:
        print(f"[CLI] Processing video file: {args.video_path}")
diff --git a/backend/providers/gemini.py b/backend/providers/gemini.py
index 2ea0834..428bca8 100644
--- a/backend/providers/gemini.py
+++ b/backend/providers/gemini.py
@@ -2,6 +2,7 @@ import json
    import logging
    import os
    import time
+from datetime import datetime, timedelta, timezone
    from typing import Tuple

    from google import genai
@@ -40,6 +41,48 @@ class GeminiProvider(AIVideoProvider):
    except Exception:
        pass

```

```

+ def cleanup_orphaned_files(self, grace_sec: int = 3600) -> int:
+     """Sweep and delete video files orphaned on the Gemini File API by a previous
+     crashed run (a hard process death between ``files.upload`` and ``_safe_delete``
+     leaves the uploaded clip behind forever ? the per-call cleanup only covers paths
+     that reach a ``finally``/except).
+
+     Best-effort: lists every file, deletes any whose ``mime_type`` is a video AND whose
+     creation is older than ``grace_sec`` (so we never race a concurrent worker mid-upload).
+     Never raises; returns the count deleted. Intended to run once at startup.
+
+     Args:
+         grace_sec: only delete files older than this (default 1h). Protects a file a
+         live worker may still be processing.
+     """
+     cutoff = datetime.now(timezone.utc) - timedelta(seconds=grace_sec)
+     deleted = 0
+     try:
+         files = list(self.client.files.list())
+     except Exception as e:
+         logger.warning("[Gemini GC] failed to list remote files (skipping sweep): %s", e)
+         return 0
+     for f in files:
+         try:
+             mime = getattr(f, "mime_type", "") or ""
+             if not mime.startswith("video/"):
+                 continue
+             # f.create_time is an RFC3339 string on the File API.
+             ct = getattr(f, "create_time", None)
+             if ct is None:
+                 # No timestamp ? can't prove it's stale; leave it (safe default).
+                 continue
+             created = datetime.fromisoformat(str(ct).replace("Z", "+00:00"))
+             if created > cutoff:
+                 continue # too recent ? a live worker may own it
+             self.client.files.delete(name=f.name)
+             deleted += 1
+         except Exception as e:
+             logger.debug("[Gemini GC] skipping a file during sweep: %s", e)
+     if deleted:
+         logger.info("[Gemini GC] deleted %d orphaned video file(s) older than %ds.",
+ deleted, grace_sec)
+     return deleted
+
+ def analyze_video(self, video_path: str, prompt: str,
+     chunk_duration: float = 0.0,
+     label: str = "") -> Tuple[list, dict]:

```

6. assistant (2026-06-25T16:54:41.839Z)

Now let me read the full gemini.py to understand the upload/analyze flow and ``\_safe\_delete``.

7. user (2026-06-25T16:54:42.060Z)

```

1     import json
2     import logging
3     import os
4     import time
5     from datetime import datetime, timedelta, timezone
6     from typing import Tuple
7
8     from google import genai
9     from google.genai import types
10
11     from backend.infrastructure.execution_policy import (
12         ExecutionPolicy,

```

```

13         PolicyTimeout,
14         poll_until,
15         run_bounded,
16     )
17     from backend.providers.base import AIVideoProvider
18
19     logger = logging.getLogger(__name__)
20
21     # The Gemini File API caps uploads at 2GB; stay well under it. Callers must chunk so
22     # each uploaded clip is below this. Oversize uploads are refused (not silently
23     attempted).
24     MAX_UPLOAD_MB = 500
25
26     class GeminiProvider(AIVideoProvider):
27         def __init__(self, api_key: str, model_name: str):
28             super().__init__(api_key, model_name)
29             # Initialize client lazily to avoid connection/auth issues during class
30             instantiation
31             self._client = None
32
33         @property
34         def client(self):
35             if self._client is None:
36                 self._client = genai.Client(api_key=self.api_key)
37             return self._client
38
39         def _safe_delete(self, video_file) -> None:
40             """Best-effort cleanup of an uploaded File (never raises)."""
41             try:
42                 self.client.files.delete(name=video_file.name)
43             except Exception:
44                 pass
45
46         def cleanup_orphaned_files(self, grace_sec: int = 3600) -> int:
47             """Sweep and delete video files orphaned on the Gemini File API by a previous
48             crashed run (a hard process death between ``files.upload`` and ``_safe_delete``
49             leaves the uploaded clip behind forever ? the per-call cleanup only covers paths
50             that reach a ``finally``/except).
51
52             Best-effort: lists every file, deletes any whose ``mime_type`` is a video AND
53             whose
54             creation is older than ``grace_sec`` (so we never race a concurrent worker mid-
55             upload).
56
57             Never raises; returns the count deleted. Intended to run once at startup.
58
59             Args:
60                 grace_sec: only delete files older than this (default 1h). Protects a file a
61                 live worker may still be processing.
62             """
63             cutoff = datetime.now(timezone.utc) - timedelta(seconds=grace_sec)
64             deleted = 0
65             try:
66                 files = list(self.client.files.list())
67             except Exception as e:
68                 logger.warning("[Gemini GC] failed to list remote files (skipping sweep):
69                 %s", e)
70
71             return 0
72
73         for f in files:
74             try:
75                 mime = getattr(f, "mime_type", "") or ""
76                 if not mime.startswith("video/"):
77                     continue
78                 # f.create_time is an RFC3339 string on the File API.
79                 ct = getattr(f, "create_time", None)
80                 if ct is None:

```

```

73             # No timestamp ? can't prove it's stale; leave it (safe default).
74             continue
75             created = datetime.fromisoformat(str(ct).replace("Z", "+00:00"))
76             if created > cutoff:
77                 continue # too recent ? a live worker may own it
78             self.client.files.delete(name=f.name)
79             deleted += 1
80         except Exception as e:
81             logger.debug("[Gemini GC] skipping a file during sweep: %s", e)
82     if deleted:
83         logger.info("[Gemini GC] deleted %d orphaned video file(s) older than %ds.",
deleted, grace_sec)
84     return deleted
85
86     def analyze_video(self, video_path: str, prompt: str,
87                       chunk_duration: float = 0.0,
88                       label: str = "") -> Tuple[list, dict]:
89         log_prefix = f"Gemini {label}" if label else "Gemini"
90
91         metrics: dict = {
92             "upload_time": 0.0,
93             "process_time": 0.0,
94             "gen_time": 0.0,
95         }
96
97         # 0. Refuse oversize uploads (Gemini File API limit is 2GB; we cap lower for
safety/speed)
98         try:
99             size_mb = os.path.getsize(video_path) / (1024 * 1024)
100         except OSError:
101             size_mb = 0.0
102         if size_mb > MAX_UPLOAD_MB:
103             logger.error(f"[{log_prefix}] {video_path} is {size_mb:.0f}MB >
{MAX_UPLOAD_MB}MB cap ? "
104                         f"SKIPPED WITHOUT ANALYSIS (no segments for this time range). "
105                         f"Re-encode/downscale chunks (indexing.ai_chunk_scale_width) or
chunk smaller.")
106             metrics["error"] = f"oversize: {size_mb:.0f}MB > {MAX_UPLOAD_MB}MB cap"
107             metrics["oversize_skipped"] = True
108             return [], metrics
109
110         # 1. Upload (retry transient network failures, e.g. WinError 10053 / reset
connections)
111         logger.info(f"[{log_prefix}] Uploading {video_path}...")
112         t_upload_start = time.time()
113         video_file = None
114         for attempt in range(3):
115             try:
116                 video_file = self.client.files.upload(file=video_path)
117                 metrics["upload_time"] = time.time() - t_upload_start
118                 logger.info(f"[{log_prefix}] Upload complete in
{metrics['upload_time']:.1f}s. URI: {video_file.uri}")
119                 break
120             except Exception as e:
121                 logger.warning(f"[{log_prefix}] Upload attempt {attempt + 1}/3 failed:
{e}")
122                 if attempt < 2:
123                     time.sleep(2 * (attempt + 1))
124         if video_file is None:
125             logger.error(f"[{log_prefix}] Upload failed after 3 attempts.")
126             metrics["error"] = "upload failed after 3 attempts"
127             return [], metrics
128
129         # 2. Wait for processing on Google servers ? a BOUNDED poll (a stuck file must
not hang

```

```

130             # the worker forever). Per-poll logs at DEBUG (no INFO spam); see
EXECUTION_POLICY.md.
131             t_process_start = time.time()
132             process_policy = ExecutionPolicy(poll_every_n_sec=10.0, max_wait_sec=1200.0) #
20 min cap
133
134             def _processed():
135                 nonlocal video_file
136                 video_file = self.client.files.get(name=video_file.name)
137                 return None if video_file.state.name == "PROCESSING" else video_file
138
139             try:
140                 video_file = poll_until(_processed, process_policy,
141                                         label=f"{log_prefix} processing", logger=logger)
142             except PolicyTimeout:
143                 logger.error(f"[{log_prefix}] Processing exceeded
{process_policy.max_wait_sec:.0f}s ? giving up.")
144                 self._safe_delete(video_file)
145                 metrics["error"] = f"processing timed out after
{process_policy.max_wait_sec:.0f}s"
146                 return [], metrics
147             except Exception as e:
148                 logger.error(f"[{log_prefix}] Status check failed: {e}")
149                 self._safe_delete(video_file)
150                 metrics["error"] = f"status check failed: {e}"
151                 return [], metrics
152             if video_file.state.name == "FAILED":
153                 logger.error(f"[{log_prefix}] Video processing failed:
{video_file.error.message}")
154                 self._safe_delete(video_file)
155                 metrics["error"] = f"video processing failed: {video_file.error.message}"
156                 return [], metrics
157
158             metrics["process_time"] = time.time() - t_process_start
159             logger.info(f"[{log_prefix}] Processing finished in
{metrics['process_time']:.1f}s.")
160
161             # 3. Call Gemini under a HARD per-attempt timeout (op_timeout_sec ? the hang-
killer that was
162             # missing: a generate() that never returns previously blocked forever) + retry
the 503/network
163             # FAILURE waves (capacity 503s on the flagship flash come in bursts; ~6
attempts).
164             logger.info(f"[{log_prefix}] Submitting multimodal request to
{self.model_name}...")
165             t_gen_start = time.time()
166             gen_policy = ExecutionPolicy(op_timeout_sec=180.0, max_retries=5,
backoff_base_sec=3.0)
167             raw_text = None
168
169             def _generate():
170                 return self.client.models.generate_content(
171                     model=self.model_name,
172                     contents=[video_file, prompt],
173                     config=types.GenerateContentConfig(
174                         response_mime_type="application/json", temperature=0.0),
175                 )
176
177             try:
178                 response = run_bounded(_generate, gen_policy, label=f"{log_prefix}
generate", logger=logger)
179                 raw_text = response.text
180                 metrics["gen_time"] = time.time() - t_gen_start
181                 logger.info(f"[{log_prefix}] Response received in
{metrics['gen_time']:.1f}s:\n{raw_text}")

```

```

182         except Exception as e:                                # PolicyTimeout (hang) or the last 503 ?
both end here
183             logger.error(f"[{log_prefix}] generate gave up after retries: {e}")
184             metrics["error"] = f"generate failed after retries: {e}"
185             metrics["generate_failed"] = True
186             self._safe_delete(video_file)
187
188         if not raw_text:
189             return [], metrics
190
191         # 4. Parse JSON
192         try:
193             segments = json.loads(raw_text)
194             if not isinstance(segments, list):
195                 logger.error(f"[{log_prefix}] Response is not a JSON array.")
196                 metrics["error"] = "response is not a JSON array"
197                 return [], metrics
198             return segments, metrics
199         except Exception as e:
200             logger.error(f"[{log_prefix}] JSON parse failed: {e}")
201             metrics["error"] = f"JSON parse failed: {e}"
202             return [], metrics
203

```

8. assistant (2026-06-25T16:54:46.985Z)

Now I have the full picture of the upload?process?generate lifecycle. Key observation: the worst-case lifetime of a *legitimate, live* file from upload to `\_safe\_delete` is:

- Upload: up to 3 attempts with backoff
- Processing poll: `max\_wait\_sec=1200.0` (20 min)
- Generate: `op\_timeout\_sec=180.0` × `max\_retries=5` + `backoff\_base\_sec=3.0`

Let me check the execution policy to compute the worst-case generate duration precisely, and understand whether multiple analyze calls reuse one file or upload many.

9. user (2026-06-25T16:54:47.805Z)

```

1     """Per-job execution policy: bounded, observable waits for long/flaky ops
(EXECUTION_POLICY.md P1).
2
3     Turns an unbounded blocking call (the GX010125 Gemini hang) into a bounded one:
4     - ``run_bounded`` ? a HARD per-attempt timeout (`op_timeout_sec`, the hang-killer) +
retry-with-backoff
5     for FAILURES (an exception) *and* HANGS (no return in time). A hung attempt's worker
thread is a daemon,
6     so it is abandoned, not joined ? it never blocks the process.
7     - ``poll_until`` ? a bounded poll loop for an in-progress external op
(`poll_every_n_sec` cadence,
8     `max_wait_sec` deadline).
9
10    **Testability:** ``sleep``/``monotonic`` are injected, so the whole module is unit-
tested with zero real
11    waiting (a fake clock advances instantly).
12
13    **Logging discipline** (so polling is VISIBLE but never NOISY): every poll/retry logs at
**DEBUG**; state
14    transitions (start, resolved, hung, gave-up) log at **INFO/WARNING**. INFO stays clean;
enable DEBUG on the
15    ``execution.policy`` logger to watch the poll trail.
16    """
17    from __future__ import annotations
18
19    import logging
20    import threading
21    import time as _time

```

```

22     from dataclasses import asdict, dataclass
23     from enum import Enum
24     from typing import Callable, Optional, TypeVar
25
26     T = TypeVar("T")
27
28     #: Dedicated logger so callers can dial polling visibility independently of the app's
root level.
29     LOG = logging.getLogger("execution.policy")
30
31
32     class WaitMode(str, Enum):
33         """How a job treats a long wait. ``str``-valued so it serialises straight to JSON on
the job row."""
34         WAIT_AND_RESUME = "wait_and_resume" # default: bounded wait; over budget ->
reschedule/partial (P2/P3)
35         ABORT = "abort" # one attempt; on hang/fail, stop immediately
36         BLOCK = "block" # bounded in-process blocking wait (no
reschedule)
37
38
39     class PolicyTimeout(TimeoutError):
40         """An op hung past ``op_timeout_sec`` or never became ready within
``max_wait_sec``."""
41
42
43     @dataclass(frozen=True)
44     class ExecutionPolicy:
45         """Declarative resilience knobs, attachable per job/op (JSON-serialisable; see
EXECUTION_POLICY.md)."""
46         mode: WaitMode = WaitMode.WAIT_AND_RESUME
47         poll_every_n_sec: float = 10.0
48         op_timeout_sec: float = 120.0 # HARD per-attempt deadline ? the hang-killer
that was missing
49         max_wait_sec: float = 1800.0 # total wall budget for a poll loop
50         max_retries: int = 5 # retries (=> max_retries+1 attempts) for
failures/hangs
51         backoff_base_sec: float = 3.0 # exponential: backoff_base * 2**attempt
52         on_timeout: str = "reschedule" # reschedule | partial | fail (consumed by
P2/P3)
53         resumable: bool = True
54
55         def to_dict(self) -> dict:
56             d = asdict(self)
57             d["mode"] = self.mode.value
58             return d
59
60         @classmethod
61         def from_dict(cls, d: dict) -> "ExecutionPolicy":
62             known = {f for f in cls.__dataclass_fields__} # type: ignore[attr-defined]
63             kw = {k: v for k, v in d.items() if k in known}
64             if "mode" in kw and not isinstance(kw["mode"], WaitMode):
65                 kw["mode"] = WaitMode(kw["mode"])
66             return cls(**kw)
67
68
69     #: The sane default (also what a job gets if it declares no policy).
70     DEFAULT_POLICY = ExecutionPolicy()
71
72
73     def _backoff_sec(policy: ExecutionPolicy, attempt: int) -> float:
74         """Deterministic exponential backoff (no jitter ? reproducible tests)."""
75         return policy.backoff_base_sec * (2 ** attempt)
76
77

```



```

78     def run_bounded(fn: Callable[[], T], policy: ExecutionPolicy = DEFAULT_POLICY, *,
79                     label: str = "op", logger: Optional[logging.Logger] = None,
80                     sleep: Callable[[float], None] = _time.sleep) -> T:
81         """Run ``fn()`` under a hard per-attempt timeout, retrying FAILURES and HANGS with
backoff.
82
83         Returns ``fn``'s value on success. Raises ``PolicyTimeout`` if the final attempt
hung, or re-raises the
84         last exception if the final attempt failed. ``mode=ABORT`` => a single attempt (no
retries).
85         """
86         log = logger or LOG
87         attempts = 1 if policy.mode is WaitMode.ABORT else policy.max_retries + 1
88         last_exc: Optional[BaseException] = None
89         for attempt in range(attempts):
90             box: dict = {}
91             done = threading.Event()
92
93             # box/done bound as defaults (not closed over) ? the thread is started + joined
within this
94             # same iteration, but explicit binding keeps each attempt isolated and silences
B023.
95             def _runner(_box: dict = box, _done: threading.Event = done) -> None:
96                 try:
97                     _box["val"] = fn()
98                 except BaseException as e:          # relay ANY error (incl. timeouts) to the
caller thread
99                     _box["exc"] = e
100                 finally:
101                     _done.set()
102
103             threading.Thread(target=_runner, name=f"bounded:{label}", daemon=True).start()
104             if done.wait(timeout=policy.op_timeout_sec):
105                 if "exc" in box:
106                     last_exc = box["exc"]
107                     log.info("%s: attempt %d/%d failed: %s", label, attempt + 1, attempts,
last_exc)
108                 else:
109                     if attempt:
110                         log.info("%s: succeeded on attempt %d/%d", label, attempt + 1,
attempts)
111                     return box["val"]                # type: ignore[return-value]
112             else:
113                 last_exc = PolicyTimeout(f"{label} hung >
op_timeout_sec={policy.op_timeout_sec}s")
114                 log.warning("%s: attempt %d/%d HUNG (>%.0fs) ? abandoning", label, attempt +
1, attempts,
115                             policy.op_timeout_sec)
116                 if attempt < attempts - 1:
117                     wait = _backoff_sec(policy, attempt)
118                     log.debug("%s: backing off %.1fs before retry", label, wait)
119                     sleep(wait)
120                 assert last_exc is not None
121                 log.warning("%s: gave up after %d attempt(s): %s", label, attempts, last_exc)
122                 raise last_exc
123
124
125     def poll_until(check: Callable[[], bool], Optional[T], policy: ExecutionPolicy =
DEFAULT_POLICY, *,
126                    label: str = "op", logger: Optional[logging.Logger] = None,
127                    sleep: Callable[[float], None] = _time.sleep,
128                    monotonic: Callable[[], float] = _time.monotonic) -> T:
129         """Poll ``check()`` every ``poll_every_n_sec`` until it returns a non-``None``
value, or raise
130         ``PolicyTimeout`` after ``max_wait_sec``. ``check`` returns ``None`` while the op is

```

```

still in progress.
131
132     Polls log at DEBUG (no INFO spam); the wait start + resolution + give-up log at
INFO/WARNING.
133     """
134     log = logger or LOG
135     start = monotonic()
136     polls = 0
137     log.info("%s: waiting (poll_every=%.0fs, max_wait=%.0fs)",
138             label, policy.poll_every_n_sec, policy.max_wait_sec)
139     while True:
140         result = check()
141         if result is not None:
142             log.info("%s: ready after %d poll(s) / %.0fs", label, polls, monotonic() -
start)
143             return result
144         elapsed = monotonic() - start
145         if elapsed >= policy.max_wait_sec:
146             log.warning("%s: still not ready after %.0fs (max_wait) ? giving up", label,
elapsed)
147             raise PolicyTimeout(f"{label} not ready within
max_wait_sec={policy.max_wait_sec}s")
148             polls += 1
149             log.debug("%s: poll %d, %.0fs elapsed, not ready", label, polls, elapsed)
150             sleep(policy.poll_every_n_sec)
151

```

10. assistant (2026-06-25T16:54:52.961Z)

Now I can compute the worst-case live lifetime precisely. For the generate step with
`op\_timeout\_sec=180.0`, `max\_retries=5` (6 attempts), `backoff\_base\_sec=3.0`:

- 6 attempts × 180s = 1080s
- backoff between: $3 \times (2^0 + 2^1 + 2^2 + 2^3 + 2^4) = 3 \times 31 = 93s$
- Generate worst case ? 1173s (~19.6 min)

Plus processing poll 1200s (20 min), plus upload retries (~6s). **Total worst-case live lifetime
? 40 min**, which is under the 1h grace. Good ? but this is a single `analyze\_video` call. Let
me check how the caller uses this ? whether one job uploads many chunks sequentially, and how
create_time relates to the whole job.

Let me find the callers of `analyze\_video` and how chunks are processed.

11. user (2026-06-25T16:54:54.543Z)

```

Found 20 files
tests\test_provider_gemini.py
backend\main.py
backend\providers\gemini.py
tests\test_trajectory_hybrid_equivalence.py
tests\test_yolo_hybrid.py
tests\test_gemini_refine.py
docs\CODE_CLEANUP_AUDIT_2026-06-24.md
backend\pipeline\segmenters\yolo_hybrid.py
backend\eval\gemini_refine.py
backend\pipeline\segmenters\trajectory_hybrid.py
tests\test_output_base.py
tests\test_gemini.py
tests\test_fusion_hybrid.py
docs\archives\past_projects\DECOUPLED_COMPUTE\05-COST-ATTRIBUTION.md
docs\CODE_MAP.md
docs\BACKLOG.md
backend\providers\base.py
backend\pipeline\segmenters\gemini.py
docs\EXECUTION_POLICY.md
tests\test_tracknet_hybrid.py

```

12. assistant (2026-06-25T16:54:57.121Z)

Let me look at how the segmenter drives multiple chunk uploads ? to understand whether a single job holds many concurrent or sequential uploads and the per-file lifetime.

13. user (2026-06-25T16:54:59.848Z)

```
1     import concurrent.futures
2     import logging
3     import os
4     import subprocess
5     import time
6     from typing import Any, Callable, Dict, List, Tuple
7
8     from backend.pipeline.segmenters.base import BasePlaySegmenter, segmenter_registry
9     from backend.providers import provider_registry
10    from backend.reporting import run_signals
11    from backend.results import (
12        Failure, # Finding B: an unrunnable config (missing key/unknown sport/provider) ? a
loud Failure, not a silent 0-rally "success"
13    )
14    from backend.sports import sport_registry
15    from backend.utils.ffmpeg import ffmpeg_bin # P2a: single ffmpeg/ffprobe resolver
16    from backend.utils.paths import output_base
17    from backend.utils.video import extract_video_chunk, get_video_duration
18
19    logger = logging.getLogger(__name__)
20
21
22    def _chunk_workdir(video_id: str, base: str = "output") -> str:
23        """Per-video chunk staging dir under <base>/chunks/<video_id> (`base` default
24        ``"output"``
25        keeps the prior literal byte-identical; the caller passes the configured output base
26        staging follows --output-dir ? COMPUTE_DECOUPLED_SERVING M3).
27
28        Concurrency isolation: two runs for DIFFERENT videos must not share one chunk dir ?
29        otherwise one run finishing (and rmdir-ing the shared dir) can undo the other's
30        in-flight work. A per-video subdir means each run only ever creates/cleans its own.
31        """
32        return os.path.join(base, "chunks", video_id)
33
34    def _parse_time(val) -> float:
35        """Converts a timestamp value to total float seconds.
36        Handles float/int pass-through, plain numeric strings, and
37        colon-separated formats (MM:SS, HH:MM:SS). Logs warnings
38        when unexpected formats are encountered."""
39        if isinstance(val, (int, float)):
40            return float(val)
41        if isinstance(val, str):
42            val = val.strip()
43            if ":" in val:
44                # Gemini returned MM:SS or HH:MM:SS despite being asked for decimal seconds.
45                # Convert it, but warn so the issue is visible in logs.
46                parts = val.split(":")
47                parts.reverse()
48                total = 0.0
49                for i, p in enumerate(parts):
50                    try:
51                        total += float(p) * (60 ** i)
52                    except ValueError:
53                        pass
54                logger.warning(f"Timestamp '{val}' was in colon-separated format
55        (MM:SS/HH:MM:SS) instead of decimal seconds. Converted to {total:.2f}s.")
```

```

55         return total
56     try:
57         return float(val)
58     except ValueError:
59         logger.warning(f"Could not parse timestamp string '{val}' as a number.
Defaulting to 0.0.")
60         return 0.0
61     logger.warning(f"Unexpected timestamp type {type(val).__name__} for value '{val}'.
Defaulting to 0.0.")
62     return 0.0
63
64
65     class GeminiPlaySegmenter(BasePlaySegmenter):
66         @property
67         def name(self) -> str:
68             return "gemini"
69
70         @property
71         def description(self) -> str:
72             return "High-precision AI-powered semantic play segment detection using Google
Gemini Multimodal API."
73
74         def _resolve_provider(self, model_name: str):
75             """Resolve the sport profile, provider, and API key (the process_video
preamble).
76
77             Returns (sport_profile, provider) on success, or a ``Failure`` (with a user-
facing
78             message) when the run cannot proceed ? an unknown sport, a MISSING API KEY, or
an
79             unknown provider. The caller propagates the Failure so the job is marked
**failed**
80             with a reason, instead of a silent 0-rally "success" that looks like an empty
match
81             (Finding B ? the loud-failures bar). A genuine "ran, found nothing" is
unaffected.
82             """
83             # Get sport profile config (config is always typed AppConfig ?
_normalize_config()
84             # in base.py converts plain dicts passed by tests at init time)
85             sport_name = self.config.sport
86             try:
87                 sport_profile = sport_registry.get(sport_name)
88             except KeyError as e:
89                 logger.error(str(e))
90                 return Failure(str(e))
91
92             # Get AI provider and model config
93             provider_name = self.config.ai_provider
94
95             # Get appropriate API key based on the provider
96             api_key_env_var = f"{provider_name.upper()}_API_KEY"
97             api_key = os.environ.get(api_key_env_var)
98             if not api_key:
99                 from backend.pipeline.segmenters._keyhint import api_key_hint
100                 msg = (
101                     f"{api_key_env_var} is not set ? the {provider_name} segmenter cannot
run. "
102                     f"Set your API key. " + api_key_hint(api_key_env_var)
103                 )
104                 logger.error(msg)
105                 return Failure(msg)
106
107             try:
108                 provider = provider_registry.get(provider_name, api_key=api_key,

```

```

model_name=model_name)
109         except KeyError as e:
110             logger.error(str(e))
111             return Failure(str(e))
112
113         return sport_profile, provider
114
115     def _maybe_trim_video(self, video_id: str, video_path: str) -> Tuple[str,
Callable[[], None]]:
116         """Optionally trim the video to indexing.debug_duration seconds (debug feature).
117
118         Returns (video_to_upload, cleanup_temp_files) where cleanup_temp_files removes
any
119         temp trimmed file created here.
120         """
121         # A: Check for debug_duration to trim the video first
122         debug_duration = self.config.indexing.debug_duration
123         video_to_upload = video_path
124         is_trimmed = False
125         temp_trimmed_path = None
126
127         if debug_duration:
128             logger.info(f"DEBUG FEATURE: Trimming video to first {debug_duration}
seconds for fast processing...")
129             base = output_base(self.config)
130             temp_trimmed_path = os.path.join(base, f"temp_trimmed_{video_id}.mp4")
131             os.makedirs(base, exist_ok=True)
132             if os.path.exists(temp_trimmed_path):
133                 try:
134                     os.remove(temp_trimmed_path)
135                 except Exception:
136                     pass
137
138             cmd = [
139                 ffmpeg_bin(), "-y",
140                 "-ss", "0",
141                 "-t", str(debug_duration),
142                 "-i", video_path,
143                 "-c", "copy",
144                 temp_trimmed_path
145             ]
146             logger.debug(f"Running command: {' '.join(cmd)}")
147             try:
148                 subprocess.run(cmd, check=True, stdout=subprocess.DEVNULL,
stderr=subprocess.DEVNULL)
149                 logger.info(f"Trimmed video successfully created at:
{temp_trimmed_path}")
150                 video_to_upload = temp_trimmed_path
151                 is_trimmed = True
152             except Exception as e:
153                 logger.warning(f"Failed to trim video using ffmpeg: {e}. Falling back to
full video.")
154
155         # Helper to clean up local temp files
156         def cleanup_temp_files():
157             if is_trimmed and temp_trimmed_path and os.path.exists(temp_trimmed_path):
158                 try:
159                     os.remove(temp_trimmed_path)
160                 except Exception:
161                     pass
162
163         return video_to_upload, cleanup_temp_files
164
165     def _process_chunked(
166         self,

```

```

167         video_id: str,
168         video_to_upload: str,
169         sport_profile,
170         provider,
171         video_duration: float,
172         chunk_duration: float,
173         chunk_overlap: float,
174         ai_scale_width: int,
175         chunks_dir: str,
176         cleanup_temp_files: Callable[[], None],
177     ) -> List[Dict[str, Any]]:
178         """--- CHUNKED PROCESSING --- (long videos): split into overlapping chunks,
179         analyze them in parallel, offset timestamps back to full-video coordinates."""
180         ai_segments: List[Dict[str, Any]] = []
181         step = chunk_duration - chunk_overlap
182         chunk_starts = []
183         t = 0.0
184         while t < video_duration:
185             chunk_starts.append(t)
186             t += step
187         num_chunks = len(chunk_starts)
188         logger.info(
189             f"Video is {video_duration:.1f}s ? splitting into {num_chunks} overlapping
chunks "
190             f"({chunk_duration}s each, {chunk_overlap}s overlap).")
191         )
192
193         os.makedirs(chunks_dir, exist_ok=True)
194
195         def process_single_chunk(ci: int, chunk_start: float) -> Tuple[str, list, dict]:
196             chunk_end = min(chunk_start + chunk_duration, video_duration)
197             actual_chunk_dur = chunk_end - chunk_start
198             chunk_label = f"Chunk {ci+1}/{num_chunks}"
199             chunk_path = os.path.join(chunks_dir, f"chunk_{video_id}_{ci}.mp4")
200
201             # Extract chunk with ffmpeg
202             logger.info(f"Extracting {chunk_label}: {chunk_start:.1f}s -
{chunk_end:.1f}s ({actual_chunk_dur:.1f}s)")
203             success = extract_video_chunk(
204                 video_to_upload, chunk_start, actual_chunk_dur, chunk_path,
205                 reencode=ai_scale_width > 0,
206                 scale_width=ai_scale_width if ai_scale_width > 0 else None,
207             )
208             if not success:
209                 logger.error(f"Failed to extract {chunk_label}. Skipping.")
210                 return chunk_label, [], {}
211
212             # Measure chunk size
213             try:
214                 chunk_size_mb = os.path.getsize(chunk_path) / (1024 * 1024)
215             except Exception:
216                 chunk_size_mb = 0.0
217
218             # Get prompt from sport profile
219             prompt = sport_profile.get_prompt(chunk_duration=actual_chunk_dur)
220
221             # Process this chunk through the provider
222             chunk_segments, metrics = provider.analyze_video(
223                 chunk_path, prompt, chunk_duration=actual_chunk_dur, label=chunk_label
224             )
225             metrics["file_size_mb"] = chunk_size_mb
226             metrics["segments_found"] = len(chunk_segments)
227
228             # B2: surface provider failures so callers can distinguish "API error"
229             # from "genuinely no rallies in this chunk" (both return [] today).

```

```

230         if metrics.get("error"):
231             logger.warning("[%s] provider error ? chunk %d skipped: %s",
232                           chunk_label, ci, metrics["error"])
233
234         # Offset timestamps back to full-video coordinates
235         for segment in chunk_segments:
236             if "start_time" in segment:
237                 try:
238                     segment["start_time"] = float(segment["start_time"]) +
chunk_start
239
240                 except (ValueError, TypeError):
241                     pass
242             if "end_time" in segment:
243                 try:
244                     segment["end_time"] = float(segment["end_time"]) + chunk_start
245                 except (ValueError, TypeError):
246                     pass
247
248             logger.info(f"{chunk_label} returned {len(chunk_segments)} play segments.")
249
250         # Clean up chunk file
251         try:
252             os.remove(chunk_path)
253         except Exception:
254             pass
255
256         return chunk_label, chunk_segments, metrics
257
258     # Run chunks in parallel. Workers come from indexing.ai_max_workers (same knob
259     # the hybrids honor; was hardcoded 5). ai_max_workers=1 = serial ? needed to
260     # gently ramp a cold/prepaid Gemini project, whose burst throttling returns a
261     # misleading "credits depleted" 429 under 5 simultaneous uploads.
262     all_metrics = {}
263     max_workers = max(1, int(self.config.indexing.ai_max_workers))
264     logger.info("Starting parallel processing with up to %d concurrent workers...",
max_workers)
265     t_chunking_start = time.time()
266     with concurrent.futures.ThreadPoolExecutor(max_workers=max_workers) as executor:
267         futures = [executor.submit(process_single_chunk, ci, start) for ci, start in
enumerate(chunk_starts)]
268         for future in concurrent.futures.as_completed(futures):
269             label, segments, metrics = future.result()
270             ai_segments.extend(segments)
271             if metrics:
272                 all_metrics[label] = metrics
273
274     # Oversize refusals must be LOUD: each one is a time range with NO analysis,
275     # so "0 segments / success" here would be a silent failure, not "no rallies".
276     oversize_skips = sum(1 for m in all_metrics.values() if
m.get("oversize_skipped"))
277     if oversize_skips:
278         logger.error(
279             f"{oversize_skips}/{num_chunks} chunks exceeded the provider upload cap
and were "
280             f"SKIPPED WITHOUT ANALYSIS ? segments are missing for those time ranges.
"
281             f"Set indexing.ai_chunk_scale_width (default 1280) so chunks are re-
encoded/"
282             f"downscaled before upload."
283         )
284     run_signals.incr(video_id, "oversize_clips_skipped", oversize_skips)
285
286     # generate() give-ups (per-attempt timeout / 503 bursts exhausting retries) are
the SAME
287     # class of silent failure: the chunk's time range gets NO analysis, so "0

```

```

segments" there is
287         # "we don't know", not "no rallies". Surface it loudly + count it so a partial
reel isn't
288         # mistaken for a complete one (observed live: a degraded model alias / capacity
503s dropped
289         # whole chunks while the run still "succeeded").
290         generate_fails = sum(1 for m in all_metrics.values() if
m.get("generate_failed"))
291         if generate_fails:
292             logger.error(
293                 f"{generate_fails}/{num_chunks} chunks FAILED generate (timeout/503
after retries) ? "
294                 f"those time ranges have NO analysis; the result is INCOMPLETE, not 'no
rallies'. "
295                 f"Re-run (transient capacity), or pin a healthy indexing model if the
alias is degraded."
296             )
297             run_signals.incr(video_id, "generate_failed_chunks", generate_fails)
298
299         # Print Summary Log
300         summary_lines = [
301             "",
302             "=" * 60,
303             "                GEMINI CHUNKING SUMMARY",
304             "=" * 60,
305             f" Total time: {time.time() - t_chunking_start:.1f}s",
306             f" Total chunks: {num_chunks}",
307             f" Total play segments found: {len(ai_segments)}",
308             "-" * 60,
309             f" {'Chunk':<12} | {'Size':<8} | {'Upload':<8} | {'Process':<8} |
{'Gen.':<8} | {'Segments':<8}",
310             "-" * 60,
311         ]
312         for i in range(num_chunks):
313             lbl = f"Chunk {i+1}/{num_chunks}"
314             m = all_metrics.get(lbl, {})
315             sz = f"{m.get('file_size_mb', 0):.1f}MB" if m else "N/A"
316             upl = f"{m.get('upload_time', 0):.1f}s" if m else "N/A"
317             prc = f"{m.get('process_time', 0):.1f}s" if m else "N/A"
318             gen = f"{m.get('gen_time', 0):.1f}s" if m else "N/A"
319             seg = m.get('segments_found', 0) if m else 0
320             summary_lines.append(f" {lbl:<12} | {sz:<8} | {upl:<8} | {prc:<8} | {gen:<8}
| {seg:<8}")
321         summary_lines.append("=" * 60)
322         logger.info("\n".join(summary_lines))
323
324         # Clean up local temp files
325         cleanup_temp_files()
326         try:
327             os.rmdir(chunks_dir)
328         except Exception:
329             pass
330
331         logger.info(f"All chunks processed. {len(ai_segments)} total play segments
before deduplication.")
332
333         return ai_segments
334
335     def _process_single(
336         self,
337         video_id: str,
338         video_to_upload: str,
339         sport_profile,
340         provider,
341         video_duration: float,

```



```

342         cleanup_temp_files: Callable[[], None],
343     ) -> List[Dict[str, Any]]:
344         """--- SINGLE-FILE PROCESSING (short videos) --- analyze the whole video in one
345         call."""
346         try:
347             file_sz = os.path.getsize(video_to_upload) / (1024 * 1024)
348         except Exception:
349             file_sz = 0.0
350
351         # Get prompt from sport profile
352         prompt = sport_profile.get_prompt(chunk_duration=video_duration)
353         ai_segments, metrics = provider.analyze_video(
354             video_to_upload, prompt, chunk_duration=video_duration, label="SingleFile"
355         )
356         if metrics.get("oversize_skipped"):
357             logger.error(
358                 f"The video ({file_sz:.0f}MB) exceeded the provider upload cap and was "
359                 f"SKIPPED WITHOUT ANALYSIS ? 0 segments here does NOT mean 'no
360                 rallies'."
361             )
362             run_signals.incr(video_id, "oversize_clips_skipped", 1)
363         elif metrics.get("error"):
364             logger.error("SingleFile provider error: %s", metrics["error"])
365         summary_lines = [
366             "",
367             "=" * 50,
368             "          GEMINI SINGLE FILE SUMMARY",
369             "=" * 50,
370             f" File Size: {file_sz:.1f} MB",
371             f" Upload time: {metrics.get('upload_time', 0):.1f}s",
372             f" API Processing time: {metrics.get('process_time', 0):.1f}s",
373             f" Prompt Generation time: {metrics.get('gen_time', 0):.1f}s",
374             f" Play segments found: {len(ai_segments)}",
375             "=" * 50,
376         ]
377         logger.info("\n".join(summary_lines))
378         cleanup_temp_files()
379
380         return ai_segments
381
382     def _validate_and_dedupe(self, ai_segments: List[Dict[str, Any]], video_duration:
383     float) -> List[Dict[str, Any]]:
384         """Parse timestamps, validate/clamp candidates, and resolve overlaps (keep
385         longer)."""
386         logger.info(f"Processing {len(ai_segments)} play segments through
387         validation...")
388
389         # 7. Parse timestamps and build candidate segment list
390         idx_cfg = self.config.indexing
391         min_segment_duration = idx_cfg.min_segment_duration
392         max_segment_duration = 90.0 # Singles play segments almost never exceed this
393
394         candidates = []
395         for idx, segment in enumerate(ai_segments):
396             start = _parse_time(segment.get("start_time", 0.0))
397             end = _parse_time(segment.get("end_time", 0.0))
398             if start >= end:
399                 logger.info(f"Skipping segment #{idx+1}: start_time ({start:.2f}s) >=
400                 end_time ({end:.2f}s)")
401                 continue
402             candidates.append({
403                 "idx": idx,
404                 "start": start,
405                 "end": end,
406                 "raw": segment

```

```

401         })
402
403     # 8. Post-hoc validation pass
404     logger.info(f"Running validation on {len(candidates)} candidate play
segments...")
405     validated = []
406     rejected = []
407
408     for c in candidates:
409         label = f"Segment #{c['idx']+1} [{c['start']:.2f}s - {c['end']:.2f}s]"
410
411         # A: Reject segments that start beyond the video
412         if video_duration > 0 and c["start"] >= video_duration:
413             rejected.append((label, f"start_time ({c['start']:.2f}s) is beyond video
duration ({video_duration:.1f}s)"))
414             continue
415
416         # B: Clamp end_time to video duration
417         if video_duration > 0 and c["end"] > video_duration:
418             logger.info(f"{label}: end_time ({c['end']:.2f}s) exceeds video duration
({video_duration:.1f}s). Clamping.")
419             c["end"] = video_duration
420
421         dur = c["end"] - c["start"]
422
423         # C: Reject segments shorter than min_segment_duration
424         if dur < min_segment_duration:
425             rejected.append((label, f"duration ({dur:.2f}s) is below minimum
({min_segment_duration}s)"))
426             continue
427
428         # D: Flag suspiciously long segments
429         if dur > max_segment_duration:
430             logger.warning(
431                 f"{label}: duration ({dur:.1f}s) is unusually long
(>{max_segment_duration}s). "
432                 f"This may be a detection error. Keeping but flagging."
433             )
434
435         validated.append(c)
436
437     # E: Resolve overlapping segments (keep the longer one)
438     if len(validated) > 1:
439         # Sort by start time
440         validated.sort(key=lambda r: r["start"])
441         non_overlapping = [validated[0]]
442         for curr in validated[1:]:
443             prev = non_overlapping[-1]
444             # Check if current segment overlaps with the previous one
445             if curr["start"] < prev["end"]:
446                 prev_dur = prev["end"] - prev["start"]
447                 curr_dur = curr["end"] - curr["start"]
448                 kept, dropped = (prev, curr) if prev_dur >= curr_dur else (curr,
prev)
449                 kept_label = f"Segment #{kept['idx']+1} [{kept['start']:.2f}s -
{kept['end']:.2f}s]"
450                 dropped_label = f"Segment #{dropped['idx']+1}
[{dropped['start']:.2f}s - {dropped['end']:.2f}s]"
451                 rejected.append((dropped_label, f"overlaps with {kept_label} ? kept
the longer segment"))
452                 if kept == curr:
453                     non_overlapping[-1] = curr # Replace previous with current
454                 else:
455                     non_overlapping.append(curr)
456         validated = non_overlapping

```

```

457
458         # Log validation summary
459         if rejected:
460             reject_lines = [f" ? {label}: {reason}" for label, reason in rejected]
461             logger.info(f"{len(rejected)} play segments were REJECTED:\n" +
"\n".join(reject_lines))
462             logger.info(f"{len(validated)} play segments passed validation (out of
{len(ai_segments)} from Gemini).")
463
464             stitch_cfg = self.config.stitching
465             begin_pad = stitch_cfg.begin_padding
466             end_pad = stitch_cfg.end_padding
467
468             total_highlight_dur = 0.0
469             for c in validated:
470                 total_highlight_dur += (c["end"] - c["start"]) + begin_pad + end_pad
471
472             mins = int(total_highlight_dur // 60)
473             secs = int(total_highlight_dur % 60)
474             logger.info(
475                 f"Estimated final highlight video duration: {mins:02d}:{secs:02d} "
476                 f"({total_highlight_dur:.1f}s total; includes {begin_pad}s begin +
{end_pad}s end padding per segment)"
477             )
478
479             return validated
480
481         def _populate_db(self, video_id: str, validated: List[Dict[str, Any]], model_name:
str) -> List[Dict[str, Any]]:
482             """9. Populate SQLite DB with validated play segments; returns segments_data."""
483             segments_data = []
484             for c in validated:
485                 start = c["start"]
486                 end = c["end"]
487                 segment = c["raw"]
488                 dur = end - start
489
490                 # Extract shot_count from Gemini or compute estimate if not present
491                 shot_count = segment.get("shuttle_exchange_count")
492                 if shot_count is None:
493                     shot_count = max(3, int(dur / 0.8)) # 1 shot every 0.8 seconds as
fallback estimate
494                 else:
495                     try:
496                         shot_count = int(shot_count)
497                     except Exception:
498                         shot_count = max(3, int(dur / 0.8))
499
500                 # Only store metrics we actually know ? no fabricated data
501                 metadata = {
502                     "shot_count": shot_count,
503                     "shot_count_confidence": segment.get("shot_count_confidence",
"Unknown"),
504                     "detector": model_name
505                 }
506
507                 segment_id = self.db.add_play_segment(video_id, start, end, None, None,
metadata)
508
509                 if segment_id:
510                     segments_data.append({
511                         "id": segment_id,
512                         "start_time": start,
513                         "end_time": end,
514                         "duration": dur,

```

```

515         "metadata": metadata
516     })
517
518     return segments_data
519
520     def process_video(self, video_id: str, video_path: str, player_pool: List[str] =
None, max_frames: int = None) -> List[Dict[str, Any]]:
521         """
522         Processes a video to detect play segments using Google Gemini Multimodal Video
API,
523         populates SQLite, and indexes detailed events.
524         """
525         model_name = self.config.ai_model
526
527         # Resolve sport profile + provider. An unrunnable config (unknown sport /
missing key /
528         # unknown provider) comes back as a Failure ? propagate it so the job is marked
FAILED
529         # with a reason, not a silent 0-rally success (Finding B).
530         resolved = self._resolve_provider(model_name)
531         if isinstance(resolved, Failure):
532             return resolved
533         sport_profile, provider = resolved
534
535         # Optionally trim the video first (debug feature); cleanup removes any temp
file.
536         video_to_upload, cleanup_temp_files = self._maybe_trim_video(video_id,
video_path)
537
538         # 1. Determine actual video duration
539         video_duration = get_video_duration(video_to_upload)
540         if video_duration > 0:
541             logger.info(f"Detected video duration: {video_duration:.1f}s")
542         else:
543             logger.warning("Could not detect video duration. Prompt will not include
duration context.")
544
545         # 2. Decide whether to use chunked processing
546         idx_cfg = self.config.indexing
547         chunk_duration = idx_cfg.chunk_duration
548         chunk_overlap = idx_cfg.chunk_overlap
549         use_chunking = video_duration > 0 and video_duration > chunk_duration
550         # Re-encode + downscale chunks for upload (0 = legacy stream-copy). Stream-
copied
551         # 4K chunks (~1.3GB/90s) exceed the provider's MAX_UPLOAD_MB and are refused, so
552         # every chunk of a 4K source gets skipped; Gemini analyzes at low res anyway.
553         ai_scale_width = int(idx_cfg.ai_chunk_scale_width or 0)
554
555         # Per-video chunk workdir (concurrency isolation): a concurrent run for another
556         # video keeps its own dir, so cleanup here never removes another run's chunks.
557         chunks_dir = _chunk_workdir(video_id, output_base(self.config))
558
559         if use_chunking:
560             ai_segments = self._process_chunked(
561                 video_id, video_to_upload, sport_profile, provider, video_duration,
562                 chunk_duration, chunk_overlap, ai_scale_width, chunks_dir,
cleanup_temp_files,
563             )
564         else:
565             ai_segments = self._process_single(
566                 video_id, video_to_upload, sport_profile, provider, video_duration,
cleanup_temp_files,
567             )
568
569         if not ai_segments:

```

```

570         return []
571
572         # Populate SQLite DB with parsed play segments
573         validated = self._validate_and_dedupe(ai_segments, video_duration)
574
575         # 9. Populate SQLite DB with validated play segments
576         segments_data = self._populate_db(video_id, validated, model_name)
577
578         return segments_data
579
580     # Register the segmenter
581     segmenter_registry.register(GeminiPlaySegmenter)
582

```

14. assistant (2026-06-25T16:55:06.139Z)

This is crucial. Chunks are processed **in parallel** (`ThreadPoolExecutor` up to ai_max_workers``), and each chunk does its own `analyze_video`` ? its own `files.upload``. So a single job uploads many files concurrently. But each file's lifetime is still bounded by the per-call worst-case (~40 min < 1h grace). However, **all** chunks share roughly the same `create_time`` (uploaded near job start), so they age together.

The critical question is whether worst-case lifetime can exceed 1h. Let me look at the `gemini_refine.py` path too, since it's another caller that uploads files, and check the config for `chunk_duration`` / generate retry parameters that could be configurable.

15. user (2026-06-25T16:55:08.563Z)

```

1      """Refine WASB rally candidates with Gemini (verify boundaries + reject false fires).
2
3      WASB (shuttle-only) over-fires on between-point service feeds. This drives the
4      EXISTING Gemini provider over the SHORT WASB candidate clips (not the whole video):
5      each candidate window is cut to a small padded clip and handed to Gemini, which ?
6      seeing the actual footage ? returns the true rally inside it (tight boundaries) or
7      nothing (it was a toss / dead time). Cheap because uploads are seconds-long clips,
8      not GB-scale chunks; high-precision because Gemini judges each candidate visually.
9
10     Reuses (no reinvented wheels): providers.GeminiProvider (via provider_registry),
11     sports.badminton prompt (sport_registry), utils.video.extract_video_chunk,
12     calibrate_wasb.make_predict_fn, export_wasb_segments.write_segments_csv.
13
14     This mirrors wasb_hybrid's Phase-3 handoff but runs standalone on a cached
15     trajectory (no WASB re-run, no DB) ? an eval/tuning driver, not the live pipeline.
16
17     Run (GEMINI_API_KEY in env):
18     python -m backend.eval.gemini_refine --video <video>.MP4 --trajectory
19     <trajectory>.csv \
20         --fps 59.94 --frame-width 1920 --out output/<video>_gemini.csv [--limit 4]
21
22     from __future__ import annotations
23
24     import argparse
25     import concurrent.futures
26     import os
27     import os.path as osp
28     import time
29     from typing import List, Optional, Tuple
30
31     from backend.eval.calibrate_wasb import BASELINE, make_predict_fn
32     from backend.eval.export_wasb_segments import TUNED, seconds_to_mmss, write_segments_csv
33     from backend.providers import provider_registry
34     from backend.sports import sport_registry
35     from backend.utils.video import extract_video_chunk, get_video_duration
36

```

```

37     Interval = Tuple[float, float]
38
39
40     def merge_overlaps(intervals: List[Interval], gap_tol: float = 0.0) -> List[Interval]:
41         """Union of overlapping intervals, optionally stitching across a small gap.
42
43         gap_tol > 0 merges two intervals separated by <= gap_tol seconds ? used to heal
44         shuttle-invisibility splits WITHIN one rally, while keeping genuine between-rally
45         (several seconds) as separate rallies. (Replaces the old envelope-per-candidate
46         which over-merged multiple real rallies WASB had bundled into one candidate.)"""
47         merged: List[Interval] = []
48         for s, e in sorted(intervals):
49             if merged and s <= merged[-1][1] + gap_tol:
50                 merged[-1] = (merged[-1][0], max(merged[-1][1], e))
51             else:
52                 merged.append((s, e))
53         return merged
54
55
56     def _offset_segments(raw: list, clip_start: float, clip_end: float) -> List[Interval]:
57         """Map a Gemini clip's returned segments back to full-video seconds, clamped to
58         clip."""
59         out: List[Interval] = []
60         for sg in raw or []:
61             try:

```

16. assistant (2026-06-25T16:55:13.246Z)

Now let me check the main.py startup ordering ? where `\_maybe\_sweep\_gemini\_orphans` is called relative to job execution, and confirm whether the server/worker spawns jobs concurrently (the per-process single-executor guard).

17. user (2026-06-25T16:55:14.147Z)

```

1300     global_config = load_app_config() # returns typed AppConfig
1301     _enforce_startup_or_exit(global_config) # M5: fail fast on an incoherent ACTIVE
1302     profile
1303     # The CLI --output-dir overrides the configured output directory. It now lives
1304     # on the typed model (OutputConfig.output_dir), so every consumer ? including
1305     # /api/compile ? reads the same value instead of a write-only runtime hack.
1306     # P0a: a RELATIVE output dir roots under the app-home (RALLY_APP_HOME); with the env
1307     # unset, resolve_output_dir("output") == "output" (CWD-relative) ? byte-identical to
1308     before.
1309     global_config.output.output_dir = resolve_output_dir(args.output_dir)
1310     os.makedirs(global_config.output.output_dir, exist_ok=True)
1311     db_path = os.path.join(global_config.output.output_dir,
1312     global_config.output.db_name)
1313     # P0e: refuse a SECOND instance against the same DB BEFORE the destructive stale-job
1314     sweep below.
1315     # The job worker is in-process, so a concurrent process opening this DB would sweep
1316     THIS one's
1317     # in-flight jobs to 'failed'. The OS lock (held for the process lifetime; auto-
1318     released on exit
1319     # or crash) is keyed to db_path, so a different --output-dir/DB still runs
1320     concurrently.
1321     global _instance_lock
1322     try:
1323         _instance_lock = acquire_lock(db_path + ".lock")
1324     except OSError as exc:

```

```

1321         # The lock path isn't writable (read-only dir / perms). Don't crash and don't
falsely claim a
1322         # second instance ? warn and proceed WITHOUT the guard (a lock-infra failure
must not block a
1323         # legitimate single instance).
1324         logger.warning("[JOBS] Could not create the single-instance lock at %s.lock
(%)"); proceeding "
1325                 "WITHOUT the second-instance guard.", db_path, exc)
1326         _instance_lock = None
1327     else:
1328         if _instance_lock is None:
1329             print(f"[FATAL] Another instance is already using this database ({db_path}).
Refusing to "
1330                 f"start ? a second instance would corrupt in-flight job state. Stop
the other one, or "
1331                 f"use a different --output-dir (RALLY_APP_HOME) to run an independent
instance.")
1332             sys.exit(1)
1333
1334         db_instance = Database(db_path)
1335
1336         # Crash recovery (#16): jobs left queued/running by a previous process are dead ?
1337         # the executor is in-process. Mark them failed so pollers see a terminal state.
1338         swept = db_instance.fail_stale_jobs()
1339         if swept:
1340             logger.warning(f"[JOBS] Swept {swept} stale job(s) from a previous run ?
failed.")
1341
1342         # O1/P11: sweep Gemini File-API uploads orphaned by a hard process death (the per-
call
1343         # _safe_delete covers normal/except paths, but a SIGKILL/OOM mid-upload leaves a
clip on
1344         # Google's servers forever). Best-effort, video-only, grace-gated; never blocks
startup.
1345         _maybe_sweep_gemini_orphans(global_config)
1346
1347         # CLI processing mode (no web UI needed). --app/--server always run the server,
never CLI single-shot.
1348         if args.video_path and not args.server and not args.app:
1349             print(f"[CLI] Processing video file: {args.video_path}")
1350             # M2: bare/local path = identity (today's behaviour); a bucket/Drive URI is
fetched to scratch.
1351             # The local-existence fast-fail only applies to a local input (stat the RESOLVED
key, not the
1352             # raw URI); a remote fetch fails loudly. An unsupported scheme is a clean error,
not a crash.
1353             storage_cfg = getattr(global_config, "storage", None)
1354             try:
1355                 input_ref = storage.resolve_input_ref(args.video_path, storage_cfg)
1356             except ValueError as e:
1357                 print(f"[FAIL] {e}")
1358                 return
1359             if input_ref.backend == "local" and not os.path.exists(input_ref.key):
1360                 print(f"[FAIL] Video file not found: {args.video_path}")
1361                 return
1362             local_video = storage.acquire_local_input(args.video_path, storage_cfg)
1363
1364             video_id = compute_video_id(local_video)
1365             was_reingest = db_instance.get_video(video_id) is not None
1366
1367             # Validate (with-context variant surfaces ffmpeg_meta for the report)
1368             print("[CLI] Validating...")
1369             val_results, val_context = registry.run_all_with_context(local_video,
global_config)
1370             failures = [r for r in val_results if not r.passed]

```

```

1371
1372         # Save video status
1373         status = "failed" if failures else "processed"
1374         db_instance.add_video(video_id, args.video_path, status, [r.model_dump() for r
in val_results])
1375
1376         seg_name = args.segmenter or getattr(getattr(global_config, "indexing", None),
"default_segmenter", "motion")
1377         segmenter_actual = None
1378         segmentation_ran = False
1379         try:
1380             print("\n" + "="*40)
1381             print("VALIDATION SUMMARY")
1382             print("="*40)
1383             for r in val_results:
1384                 status_symbol = "[PASS]" if r.passed else "[FAIL]"
1385                 print(f"{status_symbol} {r.validator_name}: {r.message}")
1386             print("="*40 + "\n")
1387
1388             if failures:
1389                 print("[FAIL] Video failed checks. Indexing halted.")

```

18. assistant (2026-06-25T16:55:19.607Z)

Important context: there IS a single-instance lock (`\_instance\_lock` keyed to `db\_path`) that refuses a second process against the **same DB**. But it is keyed by `db\_path` ? a different `--output-dir`/`RALLY\_APP\_HOME` runs concurrently and **shares the same `GEMINI\_API\_KEY` / Gemini File-API namespace**. That's the multi-process race vector the prompt asks about. Let me verify the lock semantics quickly and check whether the sweep runs before or after jobs are processed, plus whether the server starts the job worker after this point.

19. user (2026-06-25T16:55:20.728Z)

```

mypy.ini:14:# backend.api.job_worker lifted from quarantine (now takes explicit config/db
params).
scripts\install.py:8:  * creates a per-OS launcher shortcut that pins ``RALLY_APP_HOME`` to the
OS app-data dir and runs
scripts\install.py:65:  The shortcut pins ``RALLY_APP_HOME`` then runs ``indexer --app``. PURE
(no IO) so it's testable.
scripts\install.py:74:          f'set "RALLY_APP_HOME={home}"\r\n'
scripts\install.py:82:          f'export RALLY_APP_HOME="{home}"\n'
scripts\install.py:90:          f'export RALLY_APP_HOME="{home}"\n'
scripts\doctor.py:8:# NO LONGER written here ? it follows
backend.utils.app_home.default_config_path() (RALLY_APP_HOME,
scripts\doctor.py:60:  # (RALLY_APP_HOME, else CWD/config.json) ? previously this wrote to the
repo root while
backend\worker\_main_.py:100:  """M5 per-profile startup checks for the worker (the compute
process, so it probes CUDA
tests\test_api_endpoints.py:535:def test_parity_job_worker_vs_direct_cli_path(env, tmp_path,
monkeypatch):
tests\test_app_launcher.py:141:          "RALLY_APP_HOME": str(app_home), # isolate: never touch
the real OS app-data dir
tests\test_app_launcher.py:160:          assert seeded, "config.json was not seeded under
RALLY_APP_HOME"
tests\test_app_home.py:3:The load-bearing guarantee: with ``RALLY_APP_HOME`` UNSET, every
resolver is byte-identical to the
tests\test_app_home.py:18:  """Each test starts with RALLY_APP_HOME unset unless it sets it
explicitly."""
tests\test_app_home.py:53:# ---- RALLY_APP_HOME set ? everything roots there ----
tests\test_app_home.py:150:# ---- env_file_to_load: app-home .env preferred ONLY when
RALLY_APP_HOME is set ----
backend\infrastructure\database.py:113:          f"a different data directory
(RALLY_APP_HOME). DB: {self.db_path}")
backend\config\loader.py:33:# ``RALLY_APP_HOME`` (PACKAGING_PLAN.md P0a) and equals
``Path("config.json")`` (CWD-relative) when

```



```

backend\config\loader.py:36:# from a later RALLY_APP_HOME.)
tests\test_base.py:175: -
tests\test_job_worker_loop.py::test_drain_segmenter_crash_marks_failed
tests\test_billing.py:155: from backend.api import job_worker
tests\test_billing.py:164: job_worker._maybe_record_job_cost({"id": "j1"}, {"usage":
{"gpu_seconds": 180}}, cfg_off, db)
tests\test_billing.py:170: from backend.api import job_worker
tests\test_billing.py:179: job_worker._maybe_record_job_cost({"id": "j1"}, {"usage":
{"gpu_seconds": 180, "gpu_type": "L4"}}, cfg, db)
tests\test_billing.py:187: from backend.api import job_worker
tests\test_billing.py:197: job_worker._maybe_record_job_cost({"id": "j1"}, {}, cfg, db) #
result has NO 'usage' key (today's reality)
backend\utils\single_instance.py:30:def acquire_lock(lock_path: str) -> Optional[IO[str]]:
backend\main.py:21:# PACKAGING_PLAN.md P0a: prefer an app-home .env ONLY when RALLY_APP_HOME is
set (the packaged
backend\main.py:91: ``backend.main`` and ``backend.api.job_worker`` into mypy quarantine.
backend\main.py:272:# now lives in backend.api.job_worker. Re-exported here so the names stay
addressable on

```

[Showing results with pagination = limit: 30]

20. assistant (2026-06-25T16:55:25.119Z)

The single-instance lock is keyed by `db\_path`, so two instances with different `RALLY\_APP\_HOME` (different DB) run concurrently while sharing the same Gemini File-API namespace. This is the real multi-process race. Now let me look at the test file to understand what behavior was actually verified, and check the worker entrypoint to see if it also calls the sweep (or only `main.py` does).

21. user (2026-06-25T16:55:25.777Z)

```

1      import logging
2      from unittest.mock import MagicMock, patch
3
4      from backend.providers.gemini import GeminiProvider
5
6
7      def test_gemini_provider_generate_failure_is_flagged():
8          """A generate() give-up (per-attempt timeout / 503 burst after retries) must set
9              metrics['generate_failed'] so a FAILED chunk isn't silently mistaken for 'no
rallies'
10             (the silent-partial-result guard the segmenter surfaces loudly)."""
11             with patch("backend.providers.gemini.genai") as mock_genai, \
12                 patch("backend.providers.gemini.run_bounded", side_effect=RuntimeError("503
exhausted")):
13                 client_mock = MagicMock()
14                 mock_genai.Client.return_value = client_mock
15                 file_mock = MagicMock()
16                 file_mock.state.name = "ACTIVE"
17                 file_mock.name = "dummy_file"
18                 client_mock.files.upload.return_value = file_mock
19                 client_mock.files.get.return_value = file_mock
20
21                 provider = GeminiProvider(api_key="dummy_api_key",
model_name="gemini-2.5-flash")
22                 segments, metrics = provider.analyze_video("dummy.mp4", "dummy prompt",
chunk_duration=10.0)
23
24                 assert segments == []                                # no analysis for this chunk
25                 assert metrics.get("generate_failed") is True        # ...but the failure is FLAGGED,
not silent
26
27
28     def test_gemini_provider_success():
29         # Mock GenAI client

```

```

30     with patch("backend.providers.gemini.genai") as mock_genai:
31         client_mock = MagicMock()
32         mock_genai.Client.return_value = client_mock
33
34         # Mock file upload
35         file_mock = MagicMock()
36         file_mock.uri = "gs://dummy"
37         file_mock.name = "dummy_file"
38         file_mock.state.name = "ACTIVE"
39
40         client_mock.files.upload.return_value = file_mock
41         client_mock.files.get.return_value = file_mock
42
43         # Mock generate_content response
44         response_mock = MagicMock()
45         response_mock.text = '["start_time": 1.0, "end_time": 2.0]']
46         client_mock.models.generate_content.return_value = response_mock
47
48         provider = GeminiProvider(api_key="dummy_api_key",
model_name="gemini-2.5-flash")
49         segments, metrics = provider.analyze_video("dummy.mp4", "dummy prompt",
chunk_duration=10.0)
50
51         assert len(segments) == 1
52         assert segments[0]["start_time"] == 1.0
53         assert metrics["upload_time"] >= 0
54         assert metrics["process_time"] >= 0
55         assert metrics["gen_time"] >= 0
56
57         # Ensure delete is called
58         client_mock.files.delete.assert_called_with(name="dummy_file")
59
60 def test_gemini_provider_invalid_json():
61     with patch("backend.providers.gemini.genai") as mock_genai:
62         client_mock = MagicMock()
63         mock_genai.Client.return_value = client_mock
64         file_mock = MagicMock()
65         file_mock.state.name = "ACTIVE"
66         client_mock.files.upload.return_value = file_mock
67         client_mock.files.get.return_value = file_mock
68
69         response_mock = MagicMock()
70         response_mock.text = '{"not_a_list": true}'
71         client_mock.models.generate_content.return_value = response_mock
72
73         provider = GeminiProvider(api_key="dummy_api_key",
model_name="gemini-2.5-flash")
74         segments, metrics = provider.analyze_video("dummy.mp4", "dummy prompt")
75         assert len(segments) == 0
76
77 def test_gemini_provider_rejects_oversize_before_upload(monkeypatch, caplog):
78     with patch("backend.providers.gemini.genai") as mock_genai:
79         client_mock = MagicMock()
80         mock_genai.Client.return_value = client_mock
81         # 600MB > MAX_UPLOAD_MB (500) -> refuse without ever uploading
82         monkeypatch.setattr("backend.providers.gemini.os.path.getsize",
83                             lambda p: 600 * 1024 * 1024)
84         provider = GeminiProvider(api_key="k", model_name="gemini-flash-latest")
85         with caplog.at_level(logging.ERROR, logger="backend.providers.gemini"):
86             segments, metrics = provider.analyze_video("big.mp4", "p")
87         assert segments == []
88         client_mock.files.upload.assert_not_called()
89         # The refusal must be LOUD and machine-visible: callers aggregate the metrics
90         # flag into the per-video report (oversize_clip_skipped rule).
91         assert metrics["oversize_skipped"] is True

```

```

92         assert any("SKIPPED WITHOUT ANALYSIS" in r.message for r in caplog.records)
93
94
95     def test_gemini_provider_upload_fail():
96         with patch("backend.providers.gemini.genai") as mock_genai:
97             client_mock = MagicMock()
98             mock_genai.Client.return_value = client_mock
99             client_mock.files.upload.side_effect = Exception("Upload err")
100
101             provider = GeminiProvider(api_key="dummy_api_key",
102 model_name="gemini-2.5-flash")
103             segments, metrics = provider.analyze_video("dummy.mp4", "dummy prompt")
104             assert len(segments) == 0
105
106
107     # --- 01/P11: Gemini remote-file GC (startup orphan sweep) ---
108
109     def _file_mock(name, mime, create_time_iso, state="ACTIVE"):
110         f = MagicMock()
111         f.name = name
112         f.mime_type = mime
113         f.create_time = create_time_iso
114         f.state = MagicMock()
115         f.state.name = state
116         return f
117
118
119     def test_cleanup_orphaned_files_deletes_stale_videos_only():
120         """Deletes video files older than the grace window; leaves non-videos and fresh
121 files."""
122         from datetime import datetime, timedelta, timezone
123         stale = (datetime.now(timezone.utc) - timedelta(hours=2)).isoformat()
124         fresh = (datetime.now(timezone.utc) - timedelta(minutes=5)).isoformat()
125         files = [
126             _file_mock("old-vid", "video/mp4", stale),          # stale video -> deleted
127             _file_mock("fresh-vid", "video/quicktime", fresh), # fresh video -> kept (grace)
128             _file_mock("old-img", "image/png", stale),          # non-video -> kept
129             _file_mock("notts", "video/mp4", None),             # no timestamp -> kept (safe)
130         ]
131         with patch("backend.providers.gemini.genai"):
132             provider = GeminiProvider(api_key="k", model_name="m")
133             provider._client = MagicMock()
134             provider._client.files.list.return_value = iter(files)
135             deleted = provider.cleanup_orphaned_files(grace_sec=3600)
136             assert deleted == 1
137             provider._client.files.delete.assert_called_once_with(name="old-vid")
138
139     def test_cleanup_orphaned_files_list_failure_is_swallowed():
140         """A list() failure must not raise ? the sweep is best-effort and must not block
141 startup."""
142         with patch("backend.providers.gemini.genai"):
143             provider = GeminiProvider(api_key="k", model_name="m")
144             provider._client = MagicMock()
145             provider._client.files.list.side_effect = RuntimeError("network")
146             assert provider.cleanup_orphaned_files() == 0
147
148     def test_cleanup_orphaned_files_per_file_delete_failure_is_swallowed():
149         """A delete() failure on one file must not abort the sweep."""
150         from datetime import datetime, timedelta, timezone
151         stale = (datetime.now(timezone.utc) - timedelta(hours=2)).isoformat()
152         files = [
153             _file_mock("a", "video/mp4", stale),

```

```

154         _file_mock("b", "video/mp4", stale),
155     ]
156     with patch("backend.providers.gemini.genai"):
157         provider = GeminiProvider(api_key="k", model_name="m")
158         provider._client = MagicMock()
159         provider._client.files.list.return_value = iter(files)
160         # First delete raises, second succeeds ? _safe_delete swallows it.
161         provider._client.files.delete.side_effect = [RuntimeError("boom"), None]
162         deleted = provider.cleanup_orphaned_files(grace_sec=3600)
163         assert deleted == 1 # only the second file counted
164
165
166
167     # --- 01/P11: main.py startup wiring (_maybe_sweep_gemini_orphans) ---
168
169     def test_maybe_sweep_skips_non_gemini_provider(monkeypatch):
170         """A non-Gemini provider must short-circuit (no client built, no sweep)."""
171         import backend.main as main_mod
172         cfg = MagicMock()
173         cfg.ai_provider = "openai"
174         called = []
175         monkeypatch.setattr(main_mod.provider_registry, "get",
176                             lambda *a, **k: called.append(k) or MagicMock())
177         monkeypatch.setenv("GEMINI_API_KEY", "k")
178         main_mod._maybe_sweep_gemini_orphans(cfg)
179         assert called == [] # provider never instantiated
180
181
182     def test_maybe_sweep_skips_when_no_api_key(monkeypatch):
183         """No GEMINI_API_KEY -> no sweep (can't auth)."""
184         import backend.main as main_mod
185         cfg = MagicMock()
186         cfg.ai_provider = "gemini"
187         monkeypatch.delenv("GEMINI_API_KEY", raising=False)
188         called = []
189         monkeypatch.setattr(main_mod.provider_registry, "get",
190                             lambda *a, **k: called.append(k) or MagicMock())
191         main_mod._maybe_sweep_gemini_orphans(cfg)
192         assert called == []
193
194
195     def test_maybe_sweep_runs_and_swallows_errors(monkeypatch):
196         """Gemini + key present -> sweep runs; a provider error is swallowed (never blocks
197 startup)."""
198         import backend.main as main_mod
199         cfg = MagicMock()
200         cfg.ai_provider = "gemini"
201         cfg.ai_model = "gemini-2.5-flash"
202         provider = MagicMock()
203         provider.cleanup_orphaned_files.side_effect = RuntimeError("api down")
204         monkeypatch.setattr(main_mod.provider_registry, "get", lambda *a, **k: provider)
205         monkeypatch.setenv("GEMINI_API_KEY", "k")
206         # Must not raise.
207         main_mod._maybe_sweep_gemini_orphans(cfg)
208         provider.cleanup_orphaned_files.assert_called_once()

```

22. assistant (2026-06-25T16:55:29.024Z)

Now let me check the cloud worker entrypoint, which is the OTHER process that could run concurrently against the same Gemini namespace and process long videos. This is the key concurrency concern.

23. user (2026-06-25T16:55:30.037Z)

```

1      """Worker dispatcher ? pull from storage ? run the pipeline ? push outputs back.
2
3      python -m backend.worker infer --video-uri <uri> --out-uri <prefix> \
4          [--segmenter wasb_hybrid] [--config config.json] [--set k=v ...] [--scratch DIR]
5          python -m backend.worker train ...          # Gen-0 train-from-bucket (follow-up)
6
7      The URI scheme selects the storage backend via ``backend.storage`` (a bare path /
8      ``local://``
9      is an identity passthrough; ``s3://`` / ``gcs://`` is an object store), so existing
10     local runs
11     are byte-for-byte unchanged. Combine with ``RALLY_DETECTOR_IMPL=stub`` and
12     ``--set indexing.skip_ai_handoff=true`` for a **secret-free, GPU-free** run (no Gemini
13     key) ?
14     the path validated end-to-end on DigitalOcean (``docs/SETUP_NEW_MACHINE.md``).
15
16     This is a thin orchestrator: it composes the SAME building blocks the ``backend.main``
17     CLI uses
18     (validator registry ? segmenter ? report), never a forked pipeline.
19     """
20     from __future__ import annotations
21
22     import argparse
23     import csv
24     import json
25     import logging
26     import os
27     import re
28     import sys
29     import tempfile
30     from typing import Any, List
31
32     from backend import storage
33     from backend.compute import ComputeJobSpec, get_compute_backend
34     from backend.config import (
35         StartupCheckError,
36         enforce_startup_checks,
37         load_config,
38         probe_cuda_available,
39     )
40     from backend.infrastructure.database import Database
41     from backend.pipeline.segmenters.base import segmenter_registry
42     from backend.pipeline.validators.base import registry as validator_registry
43     from backend.reporting import emit_indexer_report
44     from backend.results import Failure
45     from backend.utils.ffmpeg import ffprobe_bin # P2a: single ffmpeg/ffprobe resolver
46     from backend.utils.hashing import compute_video_id
47
48     logger = logging.getLogger("backend.worker")
49
50     def _coerce(v: str) -> Any:
51         """Coerce a ``--set`` string value to bool/int/float where it round-trips, else
52         str."""
53         low = v.lower()
54         if low in ("true", "false"):
55             return low == "true"
56         for cast in (int, float):
57             try:
58                 return cast(v)
59             except ValueError:
60                 pass
61         return v
62
63     def _apply_overrides(config: Any, sets: List[str]) -> None:

```

```

61         """Apply ``a.b.c=value`` overrides onto the typed config (e.g.
indexing.skip_ai_handoff=true)."""
62         for kv in sets or []:
63             key, sep, val = kv.partition("=")
64             if not sep:
65                 raise ValueError(f"--set expects k=v, got {kv!r}")
66             parts = key.split(".")
67             obj = config
68             for p in parts[:-1]:
69                 obj = getattr(obj, p)
70             setattr(obj, parts[-1], _coerce(val))
71
72
73     def _write_intervals_csv(segments: List[dict], path: str) -> None:
74         """Decimal-second ``[start,end)`` rallies + shot count / ending reason ? the join-
friendly
75         artifact (same schema as the rally-annotator labels)."""
76         with open(path, "w", newline="") as f:
77             w = csv.writer(f)
78             w.writerow(["start", "end", "shot_count", "ending_reason"])
79             for s in segments:
80                 w.writerow([
81                     f'{float(s.get("start_time", 0.0)):.3f}',
82                     f'{float(s.get("end_time", 0.0)):.3f}',
83                     s.get("shot_count", ""),
84                     s.get("ending_reason", s.get("shot_count_confidence", "")),
85                 ])
86
87
88     def _write_report_json(video_id: str, segments: List[dict], status: str, path: str) ->
None:
89         with open(path, "w") as f:
90             json.dump({
91                 "video_id": video_id,
92                 "status": status,
93                 "segment_count": len(segments),
94                 "segments": [{"start": float(s.get("start_time", 0.0)),
95                             "end": float(s.get("end_time", 0.0))} for s in segments],
96             }, f, indent=2)
97
98
99     def _run_startup_checks(config: Any) -> bool:
100         """M5 per-profile startup checks for the worker (the compute process, so it probes
CUDA
101         best-effort). Logs warnings; returns False on a FATAL coherence error so the caller
aborts
102         cleanly (rc 2) rather than run an expensive job under an incoherent profile. A true
no-op
103         (returns True, ZERO work) when no deployment.profile is selected.
104
105         The CUDA probe is gated behind an active profile: ``probe_cuda_available()`` lazily
imports
106         torch, and pre-M5 the worker's infer/train path was torch-free (detection is
delegated to a
107         WSL/native subprocess). Probing only when a profile is set keeps the default-OFF
path a true
108         no-op of work, not just of output."""
109         profile = getattr(getattr(config, "deployment", None), "profile", "") or ""
110         cuda = probe_cuda_available() if profile else None # torch-free on the default-OFF
path
111         try:
112             enforce_startup_checks(config, cuda_available=cuda, logger=logger)
113             return True
114         except StartupCheckError:
115             return False # already logged at ERROR by enforce_startup_checks

```

```

116
117
118     def _dispatch_remote(compute: Any, args: argparse.Namespace) -> int:
119         """M6: hand ONE infer job to a remote ComputeBackend (Cloud Run) and return its rc
once the
120         outputs are durably in the bucket (the backend bucket-polls the done-marker). The
dispatched
121         container runs this SAME cmd_infer INLINE (compute_target forced local_gpu) ? never
re-dispatch.
122
123         A remote job that never writes a completion marker (a failed/hung container) makes
the bucket
124         poll exhaust its budget ? ``PolicyTimeout``. Catch it and return a CLEAN rc 4
(remote dispatch
125         timeout/failure, distinct from the local 2/3) rather than letting a raw traceback
escape."""
126         from backend.infrastructure.execution_policy import PolicyTimeout
127
128         spec = ComputeJobSpec(
129             video_uri=args.video_uri, out_uri=args.out_uri,
130             segmenter=args.segmenter, config_overrides=tuple(args.set or ()),

```

24. assistant (2026-06-25T16:55:36.655Z)

The worker dispatcher (`backend.worker`) is a SEPARATE entrypoint that does NOT call `\_maybe\_sweep\_gemini\_orphans` ? only `backend.main` does. So the sweep only runs from the main CLI/server startup. But the worker can still upload Gemini files (it runs the same segmenter pipeline). This means: a `backend.worker infer` job processing a long video could be running concurrently while a `backend.main --app` or another CLI startup fires the sweep against the shared namespace.

Now I have enough to assess all four sub-questions precisely. Let me verify one detail about the worst-case generate time and whether `op\_timeout\_sec`/`max\_retries` for generate are config-overridable (they're hardcoded in `analyze\_video`, good). Let me also confirm `create\_time` is set at upload time vs processing-completion, and check the Gemini File API default 48h auto-expiry context.

Let me verify the hardcoded gen_policy isn't config-driven and double-check there's no path where a single file lives >1h legitimately.

25. user (2026-06-25T16:55:38.353Z)

```

132:         process_policy = ExecutionPolicy(poll_every_n_sec=10.0, max_wait_sec=1200.0) # 20
min cap
140:         video_file = poll_until(_processed, process_policy,
143:         logger.error(f"[{log_prefix}] Processing exceeded
{process_policy.max_wait_sec:.0f}s ? giving up.")
145:         metrics["error"] = f"processing timed out after
{process_policy.max_wait_sec:.0f}s"
161:         # 3. Call Gemini under a HARD per-attempt timeout (op_timeout_sec ? the hang-killer
that was
166:         gen_policy = ExecutionPolicy(op_timeout_sec=180.0, max_retries=5,
backoff_base_sec=3.0)
178:         response = run_bounded(_generate, gen_policy, label=f"[{log_prefix}] generate",
logger=logger)

```

26. assistant (2026-06-25T16:55:51.104Z)

Confirmed ? the generate/process policies are hardcoded constants, not config-overridable. So the single-call worst-case lifetime is fixed at roughly: upload (~6s retry) + processing poll (1200s) + generate (6×180s + 93s backoff ? 1173s) ? **2379s ? 39.6 min**, comfortably under the 1h grace for a single chunk.

Now let me verify the key timing nuance for the multi-chunk case: all chunks are submitted near-

simultaneously at job start, so they all have `create_time` ? job start. A chunk that completes at minute 39 still has `create_time` at minute 0 ? meaning it'd be 39 min old, still under 1h. Good. But I want to verify one realistic over-1h scenario: ``ai_max_workers=1`` (serial mode mentioned in the segmenter for cold/prepaid projects). With serial processing of N chunks, the FIRST chunk uploaded gets deleted quickly, but consider a job with many chunks where later chunks upload late ? actually in serial mode, each chunk uploads only when its turn comes, so `create_time` is late too. The danger is a chunk uploaded EARLY that the worker is still holding while later chunks process. Let me re-examine: in ``analyze_video``, the file is deleted at the end of EACH call (line 186), so within one job, files don't accumulate beyond the current in-flight call(s). Per-file lifetime = single-call lifetime ? 40 min max. Good.

I now have a complete, grounded picture. Let me also quickly confirm the ``files.list()`` pagination behavior concern ? whether ``list()`` returns a pager that could be slow/large, and the `create_time` format assumption.

The genai SDK ``files.list()`` returns a ``Pager`` that auto-paginates lazily; ``list(...)`` forces full enumeration of ALL files in the project (across all videos, all concurrent jobs). On a busy shared key this could be many files and multiple network round-trips at every startup.

I have everything needed. Let me compile findings.

27. user (2026-06-25T16:56:41.465Z)

Structured output provided successfully

28. assistant (2026-06-25T16:56:59.089Z)

I've completed the safety review of PR #380. Here is a summary of what I found, grounded in the actual code.

Verdict: the core mechanism is reasonable and conservative, but the central safety claim is overstated for the

Key findings (5):

****1. MEDIUM ?** The 1h grace does NOT prevent racing a live upload across processes; it only narrows the window.** The docstring asserts it "never races a concurrent worker." That's false. The single-instance lock (``backend/main.py:1319``) is keyed to ``db_path``, so a second instance under a different ``RALLY_APP_HOME`` runs concurrently ? and ``backend.worker`` (``backend/worker/___main__.py``) is a separate entrypoint that runs the same Gemini pipeline, never takes that lock, and never sweeps. All share one Gemini File-API namespace. So process B's startup sweep can delete a file process A is still using once it ages past 1h, and A's next ``files.get`/`generate_content`` fails ? the chunk's rallies silently drop from the reel.

****2. MEDIUM ?** Worst-case legitimate single-call lifetime is ~40 min, leaving only ~20 min margin, and the coupling is invisible.** ``create_time`` is stamped at upload; the file is needed until ``_safe_delete`` at the end of generate. Worst case = processing poll cap (1200s) + generate (6x180s + 93s backoff ? 1173s) ? ****39.6 min****. Safe for one process today, but ``grace_sec`` (in `gemini.py`) and the timeouts that set the floor (``max_wait_sec``, ``max_retries`` in the same file) are uncommented, uncoupled constants. Bumping a timeout later silently erodes the margin past 1h.

****3. LOW ?** ``files.list()`` eagerly enumerates the ENTIRE project namespace on every startup**, on the boot critical path, even when there are zero orphans (the common clean-shutdown case). Suggest deferring it off the boot path, stopping pagination at the first fresh file (newest-first ordering), and/or gating on a "did not exit cleanly" sentinel.

****4. NIT ?** ``create_time`` parsing** only normalizes a trailing ``Z``; a `datetime/Timestamp` object (SDK-version-dependent) would fail parsing and silently KEEP the file at DEBUG level ? safe direction but a silent GC disable.

****5. NIT ?** video-mime-only filter** means mime-less partial uploads are never reclaimed (relies on Google's 48h auto-expiry) ? correct conservative choice, just worth documenting.

The error-swallowing and best-effort design are sound (list failure ? return 0; per-file delete

failure ? continue), and the "no timestamp ? keep" and "non-video ? keep" defaults all err safe. The principal fix is to stop claiming cross-process safety and either tag uploads with a per-run owner id or gate the sweep behind an opt-in flag.

Relevant files: `C:\Users\avidu\Projects\badminton-highlight-indexer\backend\providers\gemini.py`, `C:\Users\avidu\Projects\badminton-highlight-indexer\backend\main.py`, `C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\segmenters\gemini.py`, `C:\Users\avidu\Projects\badminton-highlight-indexer\backend\infrastructure\execution\_policy.py`, `C:\Users\avidu\Projects\badminton-highlight-indexer\backend\worker\\_\_main\_\_.py`.